

My linter reported zero issues.

That *was* the issue.

147 rules against AI slop

Hardcoded secrets. SQL built by concatenation. Icon buttons with no accessible name. `async` functions that never await. Confident comments over code that does nothing.

Three tiers: **regex** over lexically-scoped source, **AST** checks, and an opt-in **TypeScript type checker**.

Then I ran it on itself

● ● ● slopless on slopless

```
$ slopless "src/**/*.ts"
```

✓ No static analysis issues detected.
Clean architecture!

Two thousand lines of TypeScript.
Nobody writes that clean.

One template literal

The engine maps every string and comment so patterns are not matched inside them. I dumped what it actually mapped:

● ● ● protected ranges – 84-character file

TemplateHead [17..23) `hi \${

Template [28..84) `;\nObject.assign(...

↑ the entire rest of the file

A missing rescan

TypeScript's scanner needs to be told when a `${...}` span closes. Without `reScanTemplateToken`, it reads the *closing* backtick as the opening of a brand new, never-terminated literal.

So one interpolated string silently switched off **every regex rule for the rest of the file**. 116 of the 147.

The rules had inverted

I built fixtures to see what the rules were really doing. They reported the harmless case and missed the real one.

WHAT THE TOOL REPORTED	BUGGY	FIXED
4 violations, all inside strings	0	4
Nothing but commented-out code	5	2

`http://` in a licence comment: reported. A hardcoded IP in a config string: invisible.

Rules got a lexical scope

The answer was not patching 116 regexes. It was letting each rule declare *where* it is allowed to match.

```
match:
```

```
  regex: \beval\(  
  scan: code          # not in comments
```

```
match:
```

```
  regex: (\d{1,3}\.){3}\d{1,3}
```

```
  scan: strings      # only in literals
```

Every rule now ships proof

Each of the 147 carries a snippet it **must flag** and one it **must ignore**. The suite fails if a rule arrives without them.

	BEFORE	AFTER
Tests	67	560
Rules with examples	0	147

Writing them found two more real bugs on their own — including a floating-promise check looking for `.catch()` on the wrong side of the call chain.

— MEASURED, NOT CLAIMED

0 → 33

findings on its own source

6,378 → 5,494

findings across 400 real npm library files —
noise down, real detections recovered

A passing check is a claim.

Not evidence.

I shipped two releases of a tool built to catch false confidence, while it was quietly producing exactly that. It never crashed. It just agreed with me.

Test that your tool can still fail.

— OPEN SOURCE

slopless

Static analysis against AI-slop patterns. Runs as a CLI, a GitHub Action, or in VS Code.

github.com/fabriziosalmi/slopless

MIT · 147 rules · zero dependencies

Fabrizio Salmi — independent security researcher, Genoa

found a false positive? open an issue — that is the point